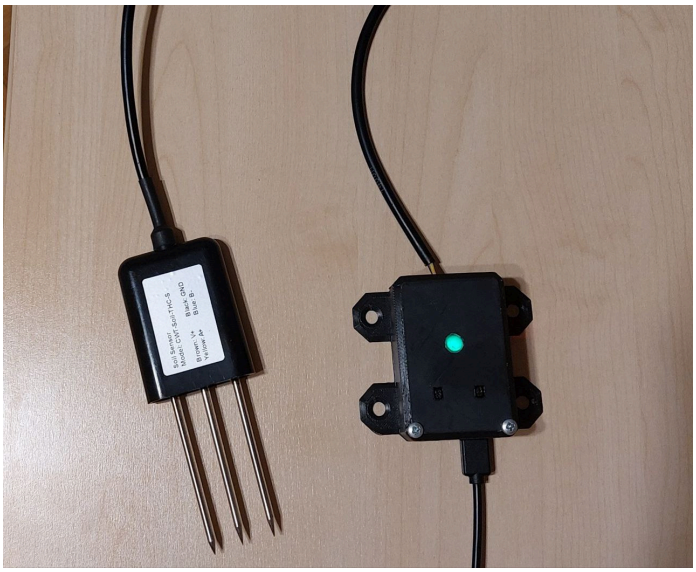
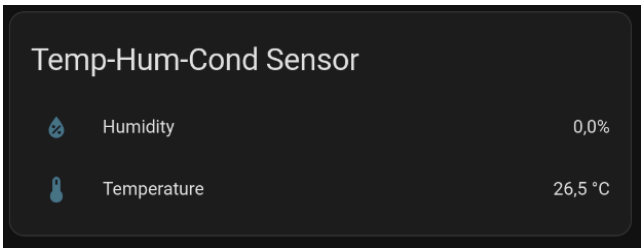
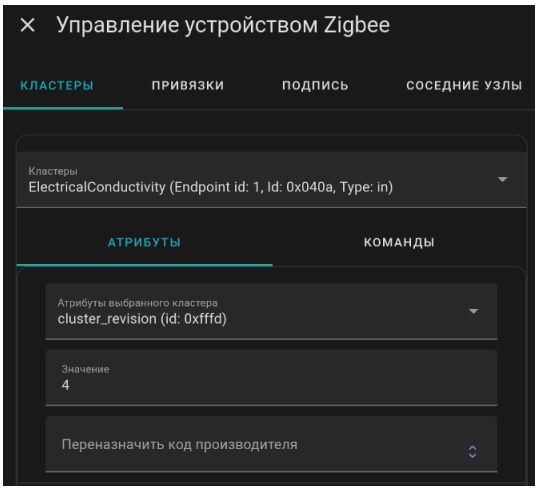
Сенсор Humidity-Temperature-Conductivity с Zigbee на базе ESP32C6



Данное устройство предназначено для измерения уровня влажности, температуры и способность почвы проводить электрический ток (т.е. показывает содержание растворенных солей в почве). Использует протокол передачи данных - Zigbee. Подключается к серверу Home-Assistant или Zigbee2MQTT через Zigbee USB-адаптер **SONOFF Zigbee 3.0 USB Dongle Plus.**





Отображение в Home-Assistant

Описание Зависимостей

Код для данного устройства опирается на гайд по созданию Zigbee-устройства для измерения CO2, Humidity, Pressure: https://habr.com/ru/articles/759964/.

Прошивка устройства состоит из 5 ключевых **компонент** (название для библиотек на ESP32).

Zigbee-компоненты из Espressif component registry:

- espressif/esp-zboss-lib – это бинарные библиотеки, реализующие низкоуровневый Zigbee стек ZBOSS для ESP32.

- espressif/esp-zigbee-lib – это библиотеки Espressif, которые предоставляют API и SDK функциональность поверх ZBOSS. Они реализуют более высокоуровневые возможности, включая поддержку стандартных кластеров Zigbee, команд, устройств домашней автоматизации и интеграцию с ESP-IDF.

Компоненты сделанные автором:

- Boot_button_conf - конфигурация кнопки Boot для кастомной настройки данной кнопки.
- Led_blink - конфигурация LED на ESP32C6 и несколько режимов работы LED. (смена цветов + режим мигания)
- Uart_rs485_soilsens - сбор данных с системы датчика CWT-Soil-THC-S + преобразователя RS485 - UART.

Также в проекте существуют следующие зависимости:

- espressif/led_strip
- Esp_driver_uart
- Esp_driver_gpio

Описание Кода

AppMain:

app_main() - задает конфигурацию драйверов и отвечает за их инициализацию. Запускает все основные таски.

```
void app_main(void)
{
  boot_button_config();
  gpio_install_isr_service(0);
  gpio_isr_handler_add(BOOT_BUTTON_PIN, button_isr_handler, NULL);
  configure_led();

  esp_zb_platform_config_t config = {
        .radio_config = ESP_ZB_DEFAULT_RADIO_CONFIG(),
        .host_config = ESP_ZB_DEFAULT_HOST_CONFIG(),
  };
  ESP_ERROR_CHECK(nvs_flash_init());
  ESP_ERROR_CHECK(esp_zb_platform_config(&config));

  activate_data_collector();
  xTaskCreate(update_attribute, "Update_attribute_value", 4096, NULL, 5, NULL);
  xTaskCreate(esp_zb_task, "Zigbee_main", 4096, NULL, 6, NULL);
}
```

Конфигурация для Boot и LED

Конфигурация для Zigbee

Основные Таски

Gpio_isr_handler_add - подключение обработчика прерываний, реагирующего на нажатие кнопки Boot.

Nvs_flash - инициализирует NVS-раздел, позволяя приложению выполнять операции чтения и записи ключ-значени. (Используется для Zigbee)

Activate_data_collector - активирует считывание данных с датчика по UART.

xTaskCreate (update_attribute, …) - Таск обновления аттрибутов Hum Temp EC на Zigbee Stack

xTaskCreate (esp_zb_task, …) - Инициализация Zigbee Stack

 # Esp_zb_task:

esp_zb_task() - Задаёт основные кластеры и аттрибуты на Zigbee Stack, определяет тип девайса (Router, Coordinator, EndPoint) и его поведение, определяет некоторые Callback-функции и инициализирует Zigbee Stack

```
/* Temperature cluster */
esp_zb_temperature_meas_cluster_cfg_t temperature_cfg = {
  .measured_value = 0,
  .min_value = 0,
  .max_value = 10000,
};

esp_zb_attribute_list_t *esp_zb_temperature_meas_cluster = esp_zb_temperature_meas_cluster_create(&temperature_cfg);
```

Пример создания и добавления кластера для температуры

```
esp_zb_ep_list_t *esp_zb_ep_list = esp_zb_ep_list_create();

esp_zb_endpoint_config_t sensor_ep_cfg =
{ .endpoint=SENSOR_ENDPOINT,
  .app_profile_id=ESP_ZB_AF_HA_PROFILE_ID,
  .app_device_id=ESP_ZB_HA_CUSTOM_ATTR_DEVICE_ID,//ESP_ZB_HA_SIMPLE_SENSOR_DEVICE_ID,
  .app_device_version=0
};
esp_zb_ep_list_add_ep(esp_zb_ep_list, esp_zb_cluster_list, sensor_ep_cfg);


esp_zb_device_register(esp_zb_ep_list);
```

Создание EndPoint со всеми кластерами на нашем устройстве.

```
esp_zb_core_action_handler_register(zb_action_handler);  Callback для команд от Zigbee-коорнатора
esp_zb_set_primary_network_channel_set(ESP_ZB_PRIMARY_CHANNEL_MASK);  Настройка сети
ESP_ERROR_CHECK(esp_zb_start(true));
esp_zb_stack_main_loop();
```

Esp_zb_start - Эта функция запускает Zigbee стек в режиме автозапуска. Она инициирует загрузку параметров из NVRAM, запуск процесса присоединения к сети (для конечных устройств — ассоциация, для координатора — формирование сети или повторное присоединение), выполняет первичную инициализацию внутренних структур и планирует callback для тестирования завершения старта.
Esp_zb_stack_main_loop - Это функция, которая запускает бесконечный главный цикл обработки событий Zigbee стека. Её вызывают после инициализации стека (esp_zb_init) и его запуска (esp_zb_start). Без постоянного

выполнения этой функции (или её эквивалента) Zigbee стек не будет функционировать полноценно, потому что задания, обработка сигналов и обмен сообщениями происходят именно внутри этого главного цикла.

 # update_attribute

Каждые 3 секунды обновляет значение аттрибутов в таблице Zigbee Stack с помощью функции esp_zb_zcl_set_attribute_val . При этом передачи данных (reporting) не происходит. Передача данных происходит либо по запросу от координатора, либо через функцию esp_zb_zcl_report_attr_cmd_req, позволяющую отправлять передачу в виде отчётов от датчика.

```
/*Data Conversion*/
humidity_int = (uint16_t) (humidity*100);
temperature_int = (uint16_t) (temperature*100);
conductivity_int = (uint16_t) conductivity;
```

Значения в ячейках переменных должны для начала масштабироваться, чтобы корректно отображаться в полях аттрибутов Zigbee Cluster

 # Esp_zb_app_signal_handler

esp_zb_app_signal_handler(esp_zb_app_signal_t *signal_struct) - callback функция, которая обрабатывает сигналы zigbee device, такие как “выход из сети”, “установление соединения”, “режим сопряжения”. Иными словами, её основное назначение — принимать

структурированный сигнал от Zigbee-стека (структура типа esp_zb_app_signal_t *signal_struct) и выполнять соответствующие действия приложения в зависимости от типа полученного сигнала.

При получении сигнала с кодом меняет цвет LED ESP32C6:

Описание	Код сигнала	Цвет
Режим сопряжения (устройство подключилось к сети)	ESP_ZB_BDB_SIGNAL_STEERING (ESP_OK)	Green
Режим сопряжения (нет подключения)	ESP_ZB_BDB_SIGNAL_STEERING (!= ESP_OK)	Blue
Выход из сети/Потеря сигнала	ESP_ZB_ZDO_SIGNAL_LEAVE, ESP_ZB_ZDO_DEVICE_UNAVAILABLE	Red

 # zb_action_handler

Это общий обработчик действий Zigbee. Он получает два параметра: `callback_id` — идентификатор типа действия, и `message` — данные события. Это позволяет централизованно обрабатывать различные события Zigbee (отчёты, ответы на команды, настройка отчетности и т.п.

 # zb_read_attr_resp_handler
Этот обработчик вызывается при получении ответа на команду Read Attribute Response. Он проверяет статус ответа, затем обходя список переменных, выводит статус, кластер, идентификатор и прочие данные атрибута. (Используется

 # zb_attribute_handler
это обработчик входящих ZCL-сообщений “установить значение атрибута”. Он вызывается стеком Zigbee, когда координатор/другой узел пишет атрибут вашему устройству или приходит отчёт о значении.

Компонент uart_rs485_soilsens:

```
// Define TX and RX pins for UART (change if needed)
#define UART_TX_PIN    5//11
#define UART_RX_PIN    4//10xTaskCreate with args

#define RECIEVER_BUFF_SIZE 16

const uart_port_t uart_num = UART_NUM_1;


void setup_uart(){
  uart_config_t uart_cfg = {
    .baud_rate = 4800,
    .data_bits = UART_DATA_8_BITS,
    .stop_bits = UART_STOP_BITS_1,
    .parity    = UART_PARITY_DISABLE,
    .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
  };

  ESP_ERROR_CHECK(uart_param_config(uart_num, &uart_cfg));
  ESP_ERROR_CHECK(uart_set_pin(uart_num, UART_TX_PIN, UART_RX_PIN, UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE));

  const int uart_buffer_size = (2*1024);
  QueueHandle_t uart_queue;
  ESP_ERROR_CHECK(uart_driver_install(uart_num , uart_buffer_size, uart_buffer_size, 10,&uart_queue, 0));

}
```

Конфигурация UART на частоте 4800 + дополнительные настройки. Использование UART_NUM1. uart_driver _install - установка драйверов UART.

Конфигурация пинов:

TX	PIN5
RX	PIN4

Чтобы отправить запрос на устройство датчика нужно предварительно отчистить буффер **uart_flush** и использовать функцию **uart_write_bytes**, а затем подождать пока вся последовательность не отправится. Ожидание отправки реализуется с помощью функции **uart_wait_tx_done.** Код запроса передаётся в массиве request длины 8 байт.

```
void send_request(uint8_t *request){
  uart_flush(uart_num);
  uart_write_bytes(uart_num, (uint8_t*)request, 8);
  ESP_ERROR_CHECK(uart_wait_tx_done(uart_num, 100/portTICK_PERIOD_MS));
}
```

Функция **sens_polling** отправляет запрос на данные и одновременно принимает данные и передает в массив сообщений.

```
void sens_polling(uint8_t* request_mess,char* data, int8_t length){
  send_request(request_mess);
  uart_read_bytes(uart_num, data, length, 100/portTICK_PERIOD_MS);
}
```

Затем сообщение обрабатывается с помощью функции **get_value_from_message** и передаётся по адресу необходимого аттрибута.

Полный цикл запроса-приёма данных находится в данном коде:

```c
void vTaskReadEC(void* dest){
  // SENSOR
  uint8_t soil_conductivity_request[] = { 0x01, 0x03, 0x00, 0x02, 0x00, 0x01, 0x25, 0xca };
  //char soil_conductivity_request[] = { 0x01, 0x03, 0x05, 0x02, 0x05, 0x01, 0x25, 0xca };
  const char* conducitvity_data_p = malloc(sizeof(char)*RECIEVER_BUFF_SIZE);
  uint32_t* conducitvity_p = (uint32_t *) dest;

  while(1){
    sens_polling(soil_conductivity_request, conducitvity_data_p, RECIEVER_BUFF_SIZE);
  // get_value<...>() waits for uint8t but gets uint32_t
    *conducitvity_p = get_value_from_message(conducitvity_data_p, RECIEVER_BUFF_SIZE);
    vTaskDelay(3500/portTICK_PERIOD_MS);
  }

  free(conducitvity_data_p);
```

Значение аттрибута передаётся в ячейку памяти (conductivity_p), адрес которой хранится в структуре данных DataPointers и передаётся в таск с помощью указателя void* dest.

Далее данные с сенсора логируются с помощью функции vTaskHtcLogs каждые 3 секунды. Логи отображаются с помощью команды [idf.py](idf.py) monitor

activate_data_collector

```c
void activate_data_collector(void)
{
  static uint8_t destination = 0;

  temperature = 5000;
  humidity = 6000;
  conductivity = 7000;

  data_p.temperature = &temperature,
  data_p.humidity=&humidity,
  data_p.conductivity = &conductivity,


  setup_uart();
  //xTaskCreate(vTaskGetSensorValue, "TaskSendData", 10000, &destination, 1, NULL );
  xTaskCreate(vTaskReadTemp, "TaskSendData", 10000, data_p.temperature, 1, NULL );
  xTaskCreate(vTaskReadHum, "TaskSendData", 10000, data_p.humidity, 1, NULL );
  xTaskCreate(vTaskReadEC, "TaskSendData", 10000, data_p.conductivity, 1, NULL );

  xTaskCreate(vTaskHtcLogs, "TaskSendLogs", 10000, &data_p, 1, NULL );

}
```

Создаём переменные для аттрибутов

Записываем адреса в структуру данных

Запускаем таски и прпедаём указатели на аттрибуты в качестве аргументов